UNIVERSIDAD DE AQUINO Bolivia

UDABOL

UNIVERSIDAD DE AQUINO BOLIVIA

PROGRAMACION AVANZADA PRÁCTICA



Ing. Kevin Arroyo Montaño

PRÁCTICA: HILOS Y MULTIPROCESOS EN PYTHON

EJERCICIO 1 — PROCESAMIENTO DE TAREAS CON HILOS

Enunciado

Una empresa necesita procesar **20 tareas de descarga**. Cada tarea simulará una operación de E/S que tarda entre **1 y 3 segundos**.

El programa debe implementar dos versiones:

Versión A — Secuencial

Las 20 tareas deben ejecutarse una después de otra.

Versión B — Con threading

Las 20 tareas deben ejecutarse utilizando **hilos**.

El programa debe mostrar:

- Número de tarea.
- Inicio de la tarea.
- Finalización.
- Tiempo individual.
- Tiempo total de ejecución.

Datos

```
tareas = [
    "Archivo_01",
    "Archivo_02",
    "Archivo_03",
    "Archivo_04",
    "Archivo_05",
    "Archivo_06",
    "Archivo_07",
    "Archivo_08",
    "Archivo_09",
    "Archivo_10",
    "Archivo_11",
```

"Archivo_12",

"Archivo_13",

"Archivo_14",

"Archivo_15",

"Archivo_16",

"Archivo_17",

"Archivo_18",

"Archivo_19",

"Archivo_20"

]

Resultado esperado

El programa debe terminar mostrando algo similar a:

==========

PROCESAMIENTO SECUENCIAL

==========

Archivo_01 → 2.01 segundos

Archivo_02 → 1.54 segundos

Archivo_03 → 2.87 segundos

...

Tiempo total: 41.23 segundos

==========

PROCESAMIENTO CON HILOS

==========

Hilo 1 → Archivo_01

Hilo 2 → Archivo_02

Hilo 3 → Archivo_03

...

Tiempo total: 2.95 segundos

Preguntas

1. ¿Cuál de las dos versiones fue más rápida?
2. ¿Por qué los hilos permiten reducir el tiempo en este caso?
3. ¿Qué ocurriría si fueran 50 tareas?
4. ¿Qué ocurriría si cada tarea tardara 10 segundos?
5. ¿Por qué este problema es adecuado para threading?

EJERCICIO 2 — PROCESAMIENTO MATEMÁTICO CON MULTIPROCESOS

Enunciado

Una empresa necesita realizar cálculos matemáticos sobre **20.000.000 de números**.

Se debe calcular:

suma de i^2

para todos los números.

El problema debe dividirse en **4 partes** y cada parte debe ser procesada de manera independiente.

Ejemplo

0 ——————————— 20.000.000

Proceso 1

0 ———— 5.000.000

Proceso 2

5.000.000 ———— 10.000.000

Proceso 3

10.000.000 ———— 15.000.000

Proceso 4

15.000.000 ———— 20.000.000

El programa debe implementar:

Versión A

Procesamiento secuencial.

Versión B

Procesamiento utilizando:

multiprocessing

Crear **4 procesos** utilizando:

multiprocessing.Process

Resultado esperado

Mostrar:

==========

PROCESAMIENTO SECUENCIAL

==========

Resultado: XXXXXXXX

Tiempo: 8.72 segundos

==========

MULTIPROCESAMIENTO

==========

Proceso 1 → terminado

Proceso 2 → terminado

Proceso 3 → terminado

Proceso 4 → terminado

Resultado: XXXXXXXX

Tiempo: 3.21 segundos

Preguntas

1. ¿Cuál versión fue más rápida?
2. ¿Por qué multiprocessing puede mejorar el rendimiento?
3. ¿Por qué este ejercicio es diferente al ejercicio 1?
4. ¿Qué relación existe entre multiprocesamiento y los núcleos de CPU?
5. ¿Qué ocurriría si utilizamos 8 procesos?

6. ¿Qué ocurre si el computador solamente tiene 2 núcleos?
7. ¿Por qué threading no sería la opción ideal para este ejercicio?